BUILD BRIEF v2

Premise audit, in-agent design and 4-day sprint plan

Meta Global AI Developer Hackathon (Cerebral Valley)
Supersedes Parts 5, 8 and 9 of the v1 document

Prepared for: CLEM **Date:** Saturday, October 31, 2026

Deadline: Wed Nov 4, 9:00 PM ET = **Thu Nov 5, 02:00 GMT**. Realistic build time left: **about 4 days**.

Correction to v1. My v1 PDF assumed today was Oct 27 and promised an 8-day runway. That was wrong: today is Oct 31, so roughly 4 build days remain. v1's schedule (Part 9) is void. v1 Parts 2 to 4 (hackathon rules, evidence base, competitor map) still hold. This document replaces the product definition, build spec and schedule, and is scoped to what is achievable in 4 days.

The one-sentence version. Build Brief becomes a thin layer that lives inside whatever coding agent you already use: it asks you the few questions that matter, turns your answers into an approved Intent Contract with *executable* acceptance checks, watches the build as it happens, and will not let the agent say 'done' until the contract is met.

A. What I understood, and how I read your raw idea

- **The flow you want:** connect Build Brief once to your coding agent (Muse Code, Codex or Claude Code). Say 'I want to build an AI invoice app'. Build Brief asks clarifying multiple-choice questions, where you can pick an option or write your own. You approve the captured intent. The agent builds. Build Brief watches, catches drift instantly, and sends corrections back, with no platform switching.
- **Why:** your landing-page rebuild took several agents, repeated prompts and screenshots. You want the first attempt to land.
- **Your insight that intent capture can be wrong** is correct and important. It means the approval step is a core feature, not a nicety.
- **My readings of unclear words** (tell me if wrong): 'MCQ' means MCP (Model Context Protocol); 'music code' means Muse Code; 'SQL or Connecta' I read as 'which integration or storage technology fits best'. Answer: MCP for tools and questions, hooks for monitoring and gating, a skill for instructions, and a local SQLite or JSON file for storage. 'The adapt in terms of the internet to context' I read as research inside the loop, handled by Muse Spark search grounding in the brief step.
- **'Train the agent to get it right in one go':** in 4 days you cannot fine-tune a model, and you should not try. The honest way to get there is better context, targeted questions and executable checks, and you prove it with a measured first-pass success rate. Details in B5.

B. Premise audit: which parts of the plan hold up

Each premise below was checked against sources found today. 'Holds' means well supported; 'Partly' means true with a design consequence; 'Fails' means do not build on it as stated.

#	Premise	Verdict	Evidence	Design consequence
1	Everything can happen inside the coding agent, no switching	Partly	Elicitation (the server pauses a tool call and asks the user a form) is documented for Claude Code, Codex CLI, Cursor and VS Code; 13 of 24 clients in one matrix support it, and Zed explicitly does not. Inline interactive UI (MCP Apps) works in Claude, VS Code Copilot, Cursor and Goose, but terminal agents are text-first. Muse Code's elicitation and MCP Apps support is not documented in anything I found.	Three-tier UX with graceful fallback: (1) native elicitation form; (2) numbered text questions the agent relays; (3) a localhost dashboard link. Never depend on one tier.
2	Asking intent questions before building improves results	Holds (strong)	Ambig-SWE (ICLR 2026): underspecified instructions hurt agents and actionable clarifying questions are the main driver of recovery. 'Asking What Matters' (2026): underspecification cut success from 43.8% to 23.7%; a model asking about 3 targeted questions recovered 88% of fully specified performance. 'Ask or Assume': an uncertainty-aware agent reached 69.4% versus 70.8% fully specified. Dialogue SWE-Bench reports existing agents ask in only 1 to 2% of interactions.	Ask few, high-value questions (aim for 3 to 6). Caveat: SWE-Clar finds clarification recovers only part of the gap for greenfield apps, and these benchmarks are mostly bug fixes, so your A/B test (B5) matters.
3	Users can answer the multiple-choice questions	Partly	The same research names 'answerability' as a separate dimension: good questions target information users actually know. Non-experts often cannot judge technical options.	Every question carries a recommended option with a one-line plain-language trade-off, a 'let Build Brief decide' choice, and 'write my own'. Ask about product facts first (who pays, which currency), technical choices last.

#	Premise	Verdict	Evidence	Design consequence
4	We can catch problems instantly while the agent builds	Partly	Muse Code runs hooks on 15 lifecycle events including UserPromptSubmit, PreToolUse, PostToolUse, Stop and SessionEnd (from Meta's own Muse Code SDK issue tracker). Codex documents 6 hook events including UserPromptSubmit and Stop. Claude Code documents about 30, and a Stop hook can block the agent from finishing and feed it a reason.	Tiered monitoring: instant deterministic checks on each edit (paths, constraints, build, tests); the LLM verdict only at end of turn. Judging every edit with a model is too slow and costly.
5	We can train a Build Brief agent to get it right in one go	Fails as stated	No fine-tuning is feasible in 4 days. But clarification (premise 2) plus executable acceptance checks are proven levers.	Reframe: no custom model. Measure 'turns to acceptance' with and without Build Brief on 5 tasks and report the difference honestly.
6	Captured intent may be wrong, so the user should approve it	Holds	Consistent with the clarification literature and with your own experience.	Intent Contract card with Approve / Edit / Add. Edits are logged and reused as examples.
7	Screenshots and visual intent matter	Holds, but risky to scope	Your landing-page story is a real pain; screenshots are how people rescue visual drift today. Comparing a rendered page to a reference needs a browser capture plus a vision model, which is flaky.	One stretch demo path only (reference screenshot, rendered screenshot, Muse Spark vision diff). Do not make it core.
8	Works across Muse Code, Codex and Claude Code	Partly	All three have MCP and hooks, but event sets and payloads differ (Codex 6 events, Muse 15, Claude Code about 30). The MCP spec also has two live versions: Codex defaults to the 2025-06-18 version while Claude Code defaults to 2026-07-28 for HTTP servers (per a third-party adapter's notes).	One small hook CLI that normalizes events from all three. Use the official MCP TypeScript SDK, which negotiates versions. Test each agent on day 1.
9	Every coding agent has an in-browser preview	Fails	Not documented as a general feature; terminal agents do not.	Ship your own localhost dashboard. Use an MCP App inline view only where a host supports it (stretch).

B5. How to honestly claim 'right in one go'

- Pick 5 greenfield tasks (an AI invoice app, a landing-page clone from a screenshot, a referral-code feature, a CSV export, a login rate limiter).
- Run each twice with the same agent: plain prompt versus Build Brief (questions, contract, checks, gate). Count turns until the acceptance checks pass, number of unrequested changes, and whether constraints were violated.
- Report the table whatever it shows. If Build Brief wins on turns and unrequested changes, that is your headline number. If it ties on some tasks, say so; judges reward measured honesty.

C. The redesigned product: the in-agent flow

```
1 CONNECT   `npx buildbrief init` detects Muse Code / Codex / Claude Code, registers the MCP
            server and the hook CLI. One command, 30 seconds.
2 INTAKE    User: 'I want to build an AI invoice app.'  -> bb_intake runs.
            Elicitation form: 3-6 questions, each = options + RECOMMENDED + one-line trade-off
            + 'write my own' + 'decide for me'.
3 CONTRACT  Intent Contract card: goals, constraints, non-goals, acceptance checks.
            User: Approve / Edit / Add.   (this is the gate against wrong intent capture)
4 COMPILE   Contract -> (a) executable acceptance checks (tests, build, lint, Playwright smoke)
            (b) protected paths (c) the Build Brief prompt injected at SessionStart.
5 WATCH     Instant (every edit): PreToolUse / PostToolUse deterministic checks.
            Per turn: Stop hook runs acceptance checks + LLM verdict (Muse Spark JSON).
            Live feed on localhost dashboard (SSE).
6 CORRECT   Stop-gate: if checks fail, the hook blocks 'done' and returns an evidence-based
            fix prompt (prompt autopsy) straight back to the agent. Max 3 loops, then ask human.
7 UNDERSTAND Short diff quiz + Comprehension Map (v1 Module 4, reduced).
```

C1. Intake questions: what to ask, in what order

Layer	Examples for 'AI invoice app'	Why this order
Product facts (answerable by anyone)	Who sends invoices (freelancers / small businesses)? Which currency and tax rules? Must invoices be exportable (PDF, CSV)?	Highest answerability; changes scope the most.
Boundaries and non-goals	What is out of scope (payments, multi-user, email sending)? Anything the agent must not touch?	Feeds the constraint guard and the unrequested-change check.
Quality bar	What must work before you call it done (create invoice, AI-fill from text, download PDF)?	Becomes the executable acceptance checks.
Technical choices (last, with a recommended default)	Stack, auth, database. Default: the stack already detected in the repo, or a simple one.	Least answerable for non-experts; default prevents stalling.

Question budget: at most 6, chosen by information value (how many files and decisions the answer changes) times answerability, with irreversible choices first. Skip any question the repo already answers (stack, framework, existing tests).

C2. Three-tier experience (so it never breaks)

Tier	Works in	What the user sees
1. Native form (MCP elicitation)	Claude Code, Codex CLI, Cursor, VS Code Copilot (documented); Muse Code unknown [verify]	A dialog with options and free-text; answers return to the tool call.
2. Text fallback	Any agent	The tool returns 'ask the user these numbered questions'; the agent relays them and passes the answers back via bb_answer.
3. Dashboard link	Any agent with a browser	A localhost page with the same questionnaire, contract card and live feed. Also the home of the ledger and Comprehension Map.

C3. Muse Code, Codex, Claude Code: capability matrix (as documented today)

Capability	Muse Code	Codex CLI	Claude Code
MCP servers	Yes (stdio, streamable HTTP)	Yes	Yes
Hooks	15 events incl. UserPromptSubmit, PreToolUse, PostToolUse, Stop	6 events incl. UserPromptSubmit, PreToolUse, PostToolUse, Stop	About 30 events; Stop can block and continue the agent

Capability	Muse Code	Codex CLI	Claude Code
Elicitation (form)	Not documented [verify]	Yes (form)	Yes (form and URL)
MCP Apps (inline UI)	Not documented	Not documented	Claude web/desktop yes; CLI is text-first
Instruction file	AGENTS.md, CLAUDE.md	AGENTS.md	CLAUDE.md

> **Strategic call:** lead with Muse Code in the demo (it scores the 25% Meta-tooling criterion), but prove portability by showing the same Intent Contract continuing in Codex or Claude Code. If Muse Code lacks elicitation, the demo uses tier 2 or 3 there and tier 1 in Claude Code, which is still honest and still impressive.

D. Outside-the-box options (ranked)

You asked for the idea you have not thought of. These are the strongest, in the order I would adopt them.

Rank	Idea	What it is	Why it is powerful	Cost in 4 days
1	Intent becomes tests (Acceptance Compiler)	Approved contract is compiled into executable checks: unit tests, build and lint rules, a Playwright smoke path, forbidden-path rules.	Turns probabilistic LLM judging into deterministic proof wherever possible. Verification is no longer 'the model thinks it matches'. Strongest answer to the Technical Depth score.	Medium. MUST.
2	Stop-gate (definition-of-done loop)	The agent cannot finish its turn until acceptance checks pass or the user overrides. A failing check returns an evidence-based fix prompt automatically.	Automates your 'repeat five times' pain, but with diagnosis instead of blind retries. Cap at 3 loops to protect cost.	Small to medium. MUST.
3	Portable intent across agents	The contract lives in the repo (.buildbrief/) and is served over MCP, so you can start in Muse Code and continue in Codex or Claude Code without re-explaining.	Directly attacks 'switching from platform to platform'. Cheap to demo, memorable.	Small. MUST.
4	Clarification budget	Choose the minimum questions with the highest information value and answerability; recommended default on every question.	Backed by the clarification research; avoids the annoying 20-question interrogation.	Small. MUST.
5	Visual intent	Reference screenshot to checklist to Playwright screenshot to Muse Spark vision diff.	Your landing-page story, solved. Great hero moment if it works.	Medium, flaky. STRETCH.
6	Intent-aware research brief	Before building, research the feature with Muse Spark search grounding and fold findings into the contract.	Removes the ChatGPT tab-switch from your original description.	Small. SHOULD.

Ideas I considered and rejected

- **Your own in-browser IDE or coding agent.** You would be competing with whole companies and lose your 4 days. Be the layer, not the agent.
- **Fine-tuning a 'Brief agent'.** Not feasible or provable in time. Use context and checks instead.
- **Mobile app.** Still no score gain. Cut.
- **Judging every file edit with an LLM.** Too slow and expensive; use deterministic checks per edit, LLM per turn.

E. Architecture v2 (small on purpose)

```
Coding agent (Muse Code / Codex / Claude Code)
  |  hooks  -> bb-hook CLI (normalizes events) --HTTP--> Build Brief daemon (Node, localhost)
  |  MCP    <------------------------------------------ MCP server (stdio or HTTP)
  |                                                      |-- intake + elicitation
  |                                                      |-- contract + check compiler
  |                                                      |-- evidence + verifier (Muse Spark JSON)
  |                                                      |-- stop-gate + autopsy
  |                                                      |-- quiz + comprehension map
  |                                                      '-- SQLite + .buildbrief/*.json (repo)
  '-- browser --> localhost dashboard (SSE live feed, contract, ledger, map)
                                  |
                                  v   Meta Model API  muse-spark-1.3  (JSON schema, 1M context, vision, search)
```

E1. Components and contracts

Component	Job	Key interface
bb-hook (CLI)	Reads the agent's hook JSON from stdin, maps to a common event, calls the daemon, returns the decision JSON (allow, block with reason, add context).	Events: session_start, prompt, pre_edit, post_edit, turn_end, session_end.
Daemon	State, checks runner, Meta API client with caching, SSE feed. Local-first; secrets read from a protected settings file (hooks run with a cleared environment on Muse Code).	HTTP localhost; SQLite.
MCP server	bb_intake, bb_answer, bb_contract, bb_verify, bb_autopsy, bb_quiz, bb_status; the ledger as an MCP resource so any agent can read it.	Official TypeScript SDK; declare elicitation use with fallback.
Check compiler	Turns contract items into runnable checks and path rules; records which items are deterministic versus model-judged.	checks.json with type: test | build | lint | path | http | model.
Dashboard	Contract, live feed, verify report, unrequested changes, quiz, Comprehension Map.	React; SSE; responsive.

E2. Core schemas

```
IntentContract { id, project, created_at, approved_at?, goals[], constraints[], non_goals[],
                 questions_answered[{q, options, chosen, source:'user'|'default'|'custom'}],
                 checks[Check], protected_paths[] }
Check { id, intent_id, type:'test'|'build'|'lint'|'path'|'http'|'model', command?, expect?,
        status:'pending'|'pass'|'fail'|'unverifiable', last_run_at, evidence }
Turn { id, session_id, prompt, hunks[], checks_run[], verdicts[], orphan_hunks[], gate:'open'|'blocked'|'override' }
Question { id, layer:'product'|'boundary'|'quality'|'technical', text,
           options[{label, tradeoff, recommended}], allow_custom:true, allow_decide_for_me:true }
```

E3. Stop-gate logic (the piece that makes it feel magic)

```
on turn_end(session):
  run deterministic checks            # build, tests, lint, path rules, http smoke
  run model verdict only for 'model' checks and orphan-hunk scan   # one Muse Spark JSON call
  if all required checks pass and no high-risk orphan hunks:  gate = open  -> allow stop
  elif loops < 3:   gate = blocked -> return { decision:'block', reason: autopsy_prompt }
  else:             gate = blocked -> ask the human (override / edit contract / abort)
autopsy_prompt = goal + failing check + evidence + what was already tried + constraints + exact acceptance
```

F. Demo script v2 (3 minutes) and score impact

Time	On screen	Voice-over beat
0:00-0:20	Stat: AI-assisted developers scored 17 points lower on comprehension; agents ask clarifying questions in only 1 to 2% of interactions.	'Agents guess. You discover the guess three prompts later.'
0:20-0:55	Muse Code: 'I want to build an AI invoice app.' A short questionnaire appears. Recommended options pre-marked; one 'write my own'. Contract card: Approve.	'Build Brief asks only what changes the outcome, then locks your intent.'
0:55-1:45	Agent builds. Live feed: edit to auth blocked by the constraint guard; a deleted test flagged as unrequested. Turn ends: 2 acceptance checks fail; stop-gate blocks 'done' and injects the fix prompt; agent fixes; checks go green.	'It cannot say done until your intent is proven. No five retries.'
1:45-2:15	Same contract continues in Claude Code or Codex.	'Switch agents without re-explaining anything.'
2:15-2:40	Diff quiz; Comprehension Map shows the invoice module amber.	'And it checks that you understood what got built.'
2:40-3:00	A/B table: turns to acceptance, unrequested changes, with and without. Meta tools used. Ledger of Build Brief's own build.	'Measured, not claimed. Built with itself.'

How v2 moves each judging criterion

Criterion	v2 effect
Impact (30%)	Targets the whole loop (ambiguity before, drift during, comprehension after), backed by clarification and comprehension research. The 'switch agents freely' story widens the audience.
Meta tooling (25%)	Muse Code plugin using hooks, MCP and skills; Muse Spark JSON schema, 1M-context verification, search grounding for the brief; optional vision for visual intent.
Demo (25%)	A single, visual story with a live block-and-fix moment. The questionnaire opens it, the stop-gate is the hero.
Technical depth (20%)	Deterministic acceptance checks, tiered monitoring, an A/B harness with real numbers, adapter layer across three agents, honest limitations.

G. The 4-day sprint (Ghana time = GMT)

Assumes one person directing several coding agents in parallel. Work packages below are split so agents do not edit the same files.

G1. Parallel work packages (one coding agent each)

Package	Owns (directories)	Delivers
WP-A Core and MCP	packages/core, packages/mcp	Schemas, intake question engine, contract compiler, MCP tools with elicitation plus text fallback.
WP-B Hooks and gate	packages/hooks, packages/daemon	bb-hook CLI for Muse Code, Codex and Claude Code; PreToolUse guard; PostToolUse checks; Stop-gate; autopsy injection.
WP-C Dashboard	apps/web	Live feed (SSE), contract card, verify report, unrequested changes, quiz, Comprehension Map.
WP-D Eval and demo	eval/, samples/	Five A/B tasks, runner, results table, sample app, demo script, README, video.

G2. Day-by-day

Day	Goal	Exit test
Sat Oct 31 (today)	Morning: run the verification checklist (H). Scaffold monorepo. WP-A: intake engine and contract; WP-B: bb-hook receiving events from Muse Code and one other agent. Start dogfooding: capture this build's own intents.	'I want to build an AI invoice app' produces questions and an approved contract in at least one agent.
Sun Nov 1	WP-A: check compiler. WP-B: constraint guard (PreToolUse) and post-edit checks. WP-C: dashboard skeleton with live feed. WP-D: write 5 task specs and acceptance checks.	An edit to a protected path is blocked live; a failing check shows on the dashboard.
Mon Nov 2	WP-B: Stop-gate and autopsy loop (cap 3). WP-A: model verdict plus orphan-hunk scan. WP-D: first A/B runs (measure cost per run). WP-C: contract card and verify report.	Stop-gate blocks 'done', injects the fix, agent passes. First A/B numbers exist.
Tue Nov 3	WP-D: finish A/B runs, results table, README, record demo (two takes). WP-C: quiz and map (or simple score list if behind). Freeze features at noon GMT.	Video v1, README and benchmark done; everything deployed and demo runs twice without edits.
Wed Nov 4	Fixes only. Re-test links in a private window. Submit by 18:00 GMT.	Submitted about 8 hours before the 02:00 GMT deadline.

G3. Cut lines (apply without debate)

- End of Sat Oct 31: if no agent shows a native questionnaire, ship the text fallback and the dashboard form, and move on.
- End of Sun Nov 1: if the check compiler produces only build and test checks, that is enough. Skip http and Playwright checks.
- End of Mon Nov 2: if the stop-gate is unstable, ship it as 'warn and suggest the fix prompt' instead of blocking.
- Tue Nov 3 noon: freeze. Drop the Comprehension Map first, then the quiz UI (keep quiz in the terminal), then visual intent.
- Always cut, never extend: onboarding tours, mobile, team accounts, notifications, MCP App inline UI (unless it takes under an hour).

H. Budget, risks and verification

H1. Money (Meta credits: $150)

Prices from third-party model listings: Muse Spark 1.3 at $1.25 per million input tokens, $0.15 per million cached input, $4.25 per million output. The numbers below are my estimates; measure your first agent run on Monday and extrapolate.

Line item	Estimate	Notes
Daily development and testing (4 days)	$25 to $40	Intake, verifier, quiz prompts, many small calls.
A/B evaluation: 5 tasks x 2 conditions, about 10 agent runs	$30 to $70	Agent builds consume far more tokens than single calls. Cap runs; use cached context. Verify the per-run cost after run 1.
Demo rehearsals and video takes	$10 to $15	
Reserve	$20 to $30	Do not spend the last $20 before submission.
Total	about $90 to $150	If run costs come in high, cut to 3 tasks (6 runs) and say so.

H2. Time (hours, one director plus several coding agents)

Block	Hours	Notes
Verification and scaffolding	3	Today morning.
Core, hooks, gate (agents in parallel)	14 to 18	Your review time is the bottleneck, not agent typing.
Dashboard	6 to 8	Keep it plain.
Eval runs and analysis	5 to 6	Mostly waiting; use that time for the README.
Demo and submission	6 to 8	Two takes, captions, link tests.

H3. Risks

Risk	Likelihood	Impact	Mitigation
Muse Code lacks native elicitation or its hook payload differs from Claude Code's	Medium	Medium	Text and dashboard fallbacks; adapter in bb-hook; test first thing today.
Stop-gate loops waste credits or annoy	Medium	High	Hard cap of 3 loops, then ask the human; log cost per loop.
Agent ignores or bypasses injected fix prompts	Medium	Medium	Return fix prompt as the hook's block reason; keep prompts short and specific.
A/B result is a tie or negative	Medium	Medium	Report honestly; emphasize unrequested-change counts and constraint violations where the gain is clearer.
Only 4 days: scope creep	High	High	Cut lines in G3. Freeze at Tue noon.
Rule issues (old code, repo, video)	Low-Med	Critical	Fresh sample app; ledger timestamps; no RiskMarshal code; confirm repo and demo-link rules in Discord.

H4. Verification checklist for the first 2 hours today

- Install Muse Code and confirm: which hook events fire, the JSON each receives, whether a Stop hook can block and return a reason, and whether MCP elicitation forms render.
- Confirm the same for Codex CLI and Claude Code with one throwaway hook that logs its input.
- Test muse-spark-1.3 from Ghana: structured JSON output, and whether it works in the same call as tools or needs a separate call.
- Read the participants-only problem statements and the official Rules (public repo, video, live link, team rules); ask in Discord if unclear.
- Decide solo versus team; every teammate must be accepted separately.

H5. Copy-paste kickoff for your coding agents

```
You are building Build Brief for the Meta Global AI Developer Hackathon. Read this PDF fully.
Hard rules: only new code written during Oct 26 - Nov 4; no secrets in the repo; Standard mode
for Meta API; deadline Nov 5 02:00 GMT; freeze features Tue Nov 3 12:00 GMT.
Product: an in-agent layer (MCP + hooks) that asks clarifying questions, compiles an approved
Intent Contract into executable acceptance checks, monitors the build, and uses a Stop-gate so
the agent cannot finish until checks pass. Support Muse Code first, then Codex and Claude Code.
Work only inside your assigned package directories. Before coding, write a short plan to
docs/plans/<package>.md and list assumptions to verify. Commit small, reference contract ids.
```

Bottom line. The raw idea is sound, and one part of it (asking before building) has strong research behind it. Two parts do not survive contact with a 4-day clock: training your own agent and depending on in-agent UI everywhere. Replace them with executable acceptance checks, a stop-gate and a measured A/B result, and you have a tighter, more defensible product than the one you described.

I. Sources (new in v2)

- Elicitation support: github.com/apify/apify-mcp-server/issues/1695 ; nango.dev/blog/mcp-elicitation-explained/ ; developersdigest.tech/blog/mcp-client-features-explained-2026 (13 of 24 clients as of Oct 18, 2026) ; github.com/Sean-Koval/hangar/pull/52 (Claude Code and Codex protocol versions) ; github.com/BlockRunAI/blockrun-mcp/pull/120 (client matrix)
- MCP Apps (inline UI): modelcontextprotocol.io/extensions/apps/overview ; pulsemcp.com/mcp-apps ; workos.com/blog/2026-01-27-mcp-apps ; strayspark.studio blog (Cursor lists MCP Apps and elicitation)
- Muse Code hooks (15 events): github.com/meta-models/muse-code-sdk/issues/49
- Codex hook events: github.com/promptconduit/cli/pull/51 ; Claude Code hooks: code.claude.com/docs/en/hooks
- Clarification research: arxiv.org/pdf/2502.13069 (Ambig-SWE, ICLR 2026) ; arxiv.org/html/2603.26233v1 and aclanthology.org/2026.findings-emnlp.15/ (Ask or Assume) ; arxiv.org/html/2604.14624v1 (Asking What Matters) ; alphaxiv.org/abs/2606.13995 (Dialogue SWE-Bench) ; arxiv.org/html/2610.31854 (SWE-Clar) ; arxiv.org/pdf/2606.29957 (SWE-Together)
- All v1 sources (hackathon rules, Meta API, comprehension debt, competitors) remain valid and are listed in the v1 document.

Several of the above are preprints or third-party summaries; read the primary text before quoting a number in your pitch.